

Chapter 4

MongoDB Query Structure

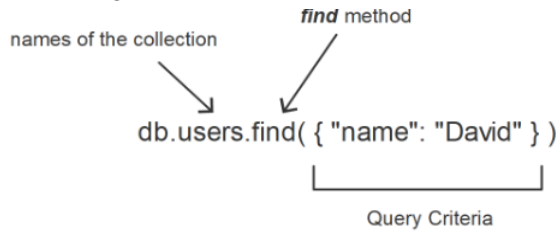


Figure 4.1: MongoDB Query Syntax

- We don't need to remember a lot of keywords and their uses.
- Due to fewer keywords, it is easier to focus on query logic. This also reduces the chance of syntactical errors.
- When the **find()** function is executed without any argument, it returns all the documents in a collection. This function can also be used with arguments.
- To return only specific documents, a condition can be provided to the **find()** function. When this is done, it evaluates it against each and every document in the collection and returns the documents that match the condition.
- The results of **find()** are enclosed in a collection, and a **cursor** to that collection is returned from the function. A **cursor is an iterator** for a collection that is used to iterate or traverse through the collection's elements.
- When the **cursor** is used from within the **mongo shell**, then it automatically iterates through the returned result collection and shows the first 20 records. However, when we use **find() from within a Programming Language, then we must manually iterate** through the returned result set using the **next()** and **hasNext()** functions.
- Eg - **db.comments.find({"name" : "Lauren Carr"})**
- The **next()** function moves the **cursor** to the next index position and returns the document from there. The **cursor's** default position is at the **beginning of the collection**. When called for the first time, the **next()** function moves the cursor to the **first document in the collection**, and that document is returned.
- To use the **cursor**, we must store it in a variable first. In the **mongo shell**, this can be done using **var comments = db.comments.find({"name" : "Lauren Carr"})**. Here, the **cursor** is stored in a variable called **comments**.
- We can then use the **cursor** to go to the next document in the collection using **comments.next()**
- When the cursor **reaches the last document in the collection**, calling **next()** will result in an error.

- The **hasnext()** function returns **true** if the collection has a document at the next index position and **false** if not.
- The **pretty()** function can be used to print a well-formatted result. Eg - **db.comments.find({"name" : "Lauren Carr"}).pretty()**
- The **findOne()** function returns only a single record. This function is useful in isolating a specific record. Its output is always well formatted since it returns only a single document.
- Eg - **db.comments.findOne()**. This query will match all documents in the collection but will only return the first document.

Choosing the Fields for the Output

- In MongoDB queries, you can either include or exclude specific fields from the result. This technique is called **projection**.
- **Projection is expressed as a second argument** to the **find()** or **findOne()** functions. In the projection expression, you can **explicitly exclude a field** by setting it to 0 **or include one** by setting it to 1.
- Eg - **db.comments.find({"name" : "Lauren Carr"}, {"name" : 1, "date": 1})**. This query returns a result with the **name**, **date** and **_id**. The **_id** is included as it is a default field present in every MongoDB document. It can be filtered by including **{"_id": 0}** inside **find()**.

3 Behaviors of Projection:

- The **_id** field will always be included, unless excluded explicitly.
- **When one or more fields are explicitly included**, the other fields (except **_id**) get excluded automatically.
- **Explicitly excluding one or more fields** will automatically include the rest of the fields, along with **_id**.

Finding The Distinct Fields:

- The **distinct()** function is used to get the distinct or unique values of a field without query criteria.
- Eg - **db.movies.distinct("rated")**. This query returns the distinct (unique) values of the **rated** field (KV pair) from each Document in the **movies** collection.
- The **distinct()** function can also be used with a query condition. Eg - **db.movies.distinct("rated", {"year" : 1994})**. This query returns the distinct ratings of all movies (movie Documents) released in 1994. Here. the first argument is the field (**rated**) and the second argument is the query(**{"year": 1994}**).

Counting The Documents:

- The **count()** function can be used to return the count of all the documents in the collection. It can also be used with an argument to return a count of the documents that match the given query.
- Eg - **db.movies.count()**.
- Without a query, this function will not physically count the documents. Instead, it will read through the collection's metadata and return the count.
- The MongoDB specification does not guarantee that the metadata count will always be accurate. Cases such as the abrupt shutdown of a database or an incomplete chunk migration in sharded collections can lead to such inaccuracy.
- When **count()** is run with a query, it does physically count all the documents that match the query's condition and return that count as a result.
- Eg - **db.mflix.count({"num_mflix_comments": 6})**.
- The **countDocuments()** function returns the count of documents that're matched by the given condition. Eg - **db.movies.countDocuments({"year": 1999})**. This query returns the count of all the movies released in 1999. The **countDocuments()** function must always be executed with a query, else it will fail.
- All the documents in the function can be counted by passing an empty query **{ }** to the **countDocuments()** function.
- The **countDocuments()** function always counts the matched documents. IT DOES NOT USE COLLECTION METADATA.
- The **estimatedDocumentCount()** function takes no query and returns the document count of the collection according to the collection metadata.

Conditional Operators:

- The **\$eq** operator stands for **equality** (**=**). It can be used by queries to find documents with fields that match a given value. Eg - **db.movies.find({"num_mflix_comments" : {\$eq : 5 }})**. This query returns all the movie Documents that have exactly 5 comments.
- The **\$ne** operator stands for **Not Equal To**. It selects all the documents where the value of the field **doesn't match** with the given value. Eg - **db.movies.find({"num_mflix_comments" : {\$ne : 5 }})**. This query returns all the movie Documents that **do not have** exactly 5 comments.
- The **\$gt** keyword can be used to find documents where the value of the field is greater than the value in the query.

- Similarly, the **\$gte** keyword is used to find documents where the value of the field is the same as or greater than the given value. Eg - **db.movies.find({year : {\$gt : 2015}}).count()**
- The **\$lt** and **\$lte** operators are used to find documents where the value of the field is less than or equal to the given value.
- The **\$in** operator finds all the documents where the field of the value matches at least one of the given values. Eg - **db.movies.find({"rated" : {\$in : ["G", "PG", "PG-13"]}})**. This will return a count of movie documents having either of the values G / PG / PG-13.
- The **\$nin** operator stands for Not In and matches all the documents where the value of the field does not match with any of the array elements.
- If **\$nin** is used with a non-existent field, then it will match all the documents. However, if **\$nin is used with null**, then it won't match any document.

Exercise:

- The number of movies the actor has acted in. Ans - **db.movies.countDocuments({"cast": "Leonardo DiCaprio"})**
- The genre of these movies. Ans - **db.movies.distinct("genre", {"cast": "Leonardo DiCaprio"})**
- Movie titles and their respective years of release. Ans - **db.movies.find({"cast": "Leonardo DiCaprio"}, {"title": 1, "year": 1})**
- The number of movies he has directed. Ans - **db.movies.countDocuments({"directors": "Leonardo DiCaprio"})**

Logical Operators:

- Using the **\$and** operator, you can have any number of conditions wrapped in an array and the operator will return only the documents that satisfy all the conditions.
- With the **\$or** operator, you can pass multiple conditions wrapped in an array and the documents satisfying either of the conditions will be returned. This operator is used when we have multiple conditions and we want to find documents that match at least one condition.
- The **\$in** operator is used to determine whether a given field has at least one of the values provided in an array, whereas the **\$or** operator is not bound to any specific fields and accepts multiple expressions.
- The **\$nor** operator is syntactically like **\$or** but behaves in the opposite way. The **\$nor** operator accepts multiple conditional expressions in the form of an array and returns the documents that do not satisfy any of the given conditions.

- The **\$not** operator represents the logical NOT operation that negates the given condition.

REFER TO EXERCISE 4.03 - COMBINING MULTIPLE QUERIES (Page 178)

- In MongoDB, regular expressions can be used with the help of the **\$regex** operator.
- To find only the strings that start with the given regular expression, the **caret operator (^)** can be used
- To find only the strings that end with the given regular expression, the **\$ operator (\$)** can be used.
- Searching with regular expressions is case-sensitive by default. The casing of the characters in the provided search pattern is matched exactly.
- The **\$options** operator allows case-insensitive regexp searches.
- Eg - **db.movies.find({"title" :{"\$regex" : "the", \$options: "i"}})**. Here, the option "i" stands for case-insensitive.

Query Arrays and Nested Documents

- When an array field is queried using a value, all those documents are returned where the array field contains at least one element that satisfies the query.
- Eg - **db.movies.find({\$and : [{"cast" : "Charles Chaplin"}, {"cast": "Edna Purviance" }] })**. This query returns all movies having "Charles Chaplin" and "Edna Purviance" in their **cast** array.
- When you search an array field using an array value, the elements and their order must match. **If the order of the array elements in the query is changed, then different documents will be matched.**
- This happens as, when array fields are searched using an array value, the value is matched using an equality check. **Any two arrays only pass the equality check if they have the same elements in the same order.**
- The **\$all** operator finds all those documents where the value of the field contains all the elements, irrespective of their order or size.

Projecting Array Elements

- You can search an array by an element value and use projection to exclude all but the first matching element of the array using the **\$** operator.
- Eg - **db.movies.find({"languages" : "Syriac"}, {"languages" :1})**. This query finds all the Documents that contain the **Syriac** language in the **languages** array.

- However, when this same query is executed as follows: **db.movies.find({"languages" : "Syriac"}, {"languages.\$" : 1})**, then with the **\$ operator** added to the projection part of the query.
- The array field in the output only contains the matching element; the rest of the elements are skipped. **Thus, the languages array in the output only contains the Syriac element.** The most important thing to remember is that **if more than one element is matched, the \$ operator projects only the first matching element.**

Projecting Array Elements By Using Their Matching Position (\$slice)

- The **\$slice** operator is used to limit the array elements based on their index position. This operator can be used with any array field, irrespective of the field being queried or not.
- Eg - **db.movies.find({"title" : "Youth Without Youth"}, {"languages" : {\$slice : 3}}).pretty()**. This query returns, in each Document, its title and the first 3 languages of the **languages** array.
- The **\$slice** operator can also accept negative indices.

Querying Nested Objects

- When nested object fields are searched with object values, there must be an exact match. This means that all the field-value pairs, along with the order of the fields, must match exactly.

Querying Nested Object Fields

- Dot notation can be used to search nested objects by providing the values of its fields. Eg - **db.movies.find({"awards.wins" : 4})**
- The nested field search is performed independently on the given fields, irrespective of the order of the elements. You can search by multiple fields and use any of the conditional or logical query operators.
- When we query using nested object fields, then only the queried nested object fields and the **_id** are included in the output. Any nested object field that we haven't queried won't be included in the output.
- A

Limiting The Result

- To limit the number of records a query returns, the resulting cursor provides a function called **limit()**. This function accepts an integer and returns the same number of records, if available.
- Eg - **db.movies.find({"cast" : "Charles Chaplin"}, {"title": 1, "_id" : 0}).limit(3)**. This query returns the 1st 3 Documents of the movies containing Charlie Chaplin in the **cast**.

- When the limit size is larger than the actual records within the cursor, all the records will be returned, irrespective of the set limit. The same will happen if the **limit()** is set to zero.
- For queries returning smaller records, a -ve size limit is equivalent to a +ve one.

Negative Limits and Batch Sizes

- MongoDB uses batches if the total number of records in the result exceeds the batch size. However, when we use a negative number to specify the limit size, only the first batch is returned and the next batch, even if it is required, will not be processed.

Sorting

- Sorting is used to return documents in a specified order. Without using explicit sorting, MongoDB does not guarantee the order in which the documents will be returned.
- The MongoDB cursor provides a **sort()** function that accepts an argument of the document type, where the document defines a sort order for specific fields